

Intelligent Conversational AI and Crime Analytics Platform

Executive summary

The right architecture for this platform is **not** a single “chatbot on top of a database.” It is a **governed decision-support system** with four distinct planes: a secure transactional **data plane** for FIR and investigation records, an **analytics plane** for graph, geospatial, and forecasting workloads, an **AI orchestration plane** for natural-language reasoning and tool use, and a **governance plane** for RBAC, auditability, evidence trails, and security monitoring. That separation is the main architectural move that allows the system to be useful to investigators while remaining defensible for supervision, legal scrutiny, and cyber resilience. The uploaded Karnataka Police ER diagram already defines a strong transactional core around `CaseMaster`, `ComplainantDetails`, `Victim`, `Accused`, `ArrestSurrender`, `Act`, `Section`, `ActSectionAssociation`, `CrimeHead`, `CrimeSubHead`, `ChargesheetDetails`, and supporting master tables; the recommended design preserves that core and adds canonical entity, graph, evidence, conversation, and forecasting layers around it.

fileciteturn0file0

For deployment, the best fit for a state crime platform is usually **hybrid with on-prem-first data residency**: keep the core crime database, identity, audit logs, graph store, search index, evidence store, and most inference inside a state data centre or government cloud, and allow only tightly controlled external model use for non-sensitive experimentation or a secondary assist path. Kubernetes is the right deployment substrate because it provides a well-understood control plane plus worker-node model for isolating stateless services from stateful data systems, and it works in both on-prem and cloud footprints. Keycloak or an equivalent OIDC provider should front all APIs and UIs, while CDC from the transactional database should publish change events into an event backbone so that graph, search, and feature stores remain near-real-time without direct cross-service database coupling. ¹

The AI stack should be **tool-first, retrieval-first, and evidence-first**. Case facts and legal sections must come from the system of record through SQL/Cypher/search tools, not from LLM parametric memory. Function calling and structured outputs are now mature enough to make the LLM act as a controlled planner that selects tools, emits JSON that adheres to schema, and returns explanations tied to database artifacts. Prompt caching and asynchronous batch jobs reduce latency and cost for repetitive prompts, nightly embeddings, offline summarisation, and backfills. ²

For model choice, use **different models for different jobs** instead of one giant model for everything. A frontier commercial model can act as the reasoning/orchestration layer; open-weight models can be self-hosted for sensitive summarisation and extraction; MuRIL, IndicNER, and IndicTrans2 are especially useful for Indian language understanding, NER, and translation; multilingual E5 is a strong retrieval embedding family; Whisper or NVIDIA Riva supports on-prem speech, while managed providers such as Google Cloud Speech-to-Text and Azure Speech have documented Kannada support; graph learning should begin with node2vec and GraphSAGE, then move to temporal GNNs only after the data pipeline is stable. ³

For proactive intelligence, the platform should support **hotspot forecasting, near-repeat analysis, criminal network analysis, and anomaly detection**, but it should not be designed as an autonomous predictive-policing engine. Crime forecasting systems are vulnerable to reporting bias and feedback loops, so outputs should be used for triage, review, and preventive planning rather than automated targeting. For high-stakes offender scoring, interpretable models are preferable to opaque black boxes, especially where decisions affect liberty, surveillance, or investigative priority. ⁴

Security and governance are first-class requirements, not add-ons. The platform should map baseline controls to NIST SP 800-53 and zero-trust principles, defend APIs against the OWASP API Security Top 10, and comply with Indian cyber obligations such as CERT-In's incident reporting, log-retention, NTP synchronisation, and point-of-contact requirements. India's IT Act and the DPDP framework are relevant, but exact applicability, exemptions, retention rules, and law-enforcement processing bases should be reviewed by counsel because public-sector and criminal-justice contexts can have special treatment and subordinate rules. ⁵

The implementation path should therefore be phased. An MVP should deliver natural-language query, bilingual text chat, FIR/case lookup, explainable summaries, evidence trails, full audit logging, and simple hotspot/network analysis. The full programme should add multilingual voice, entity resolution, financial link analysis, temporal graph analytics, forecasting, and supervisory workflow automation. A realistic estimate is roughly **30–45 person-months for a production-grade MVP** and **90–140 person-months for a full statewide platform**, depending on data quality, integrations, and whether speech and self-hosted models are included from day one. That estimate is an engineering inference for planning, not a vendor quote. ⁶

Assumptions and design principles

This report assumes that the uploaded ER diagram is the authoritative starting point for the operational FIR database and that the target environment is a **state-level law-enforcement platform** with district, unit, court, and employee hierarchies similar to the Karnataka Police schema. It also assumes that the system must support English and Kannada first, with room for additional Indian languages later. The source ER is operationally case-centric, with **CaseMaster** as the core record and one-to-many links to victims, accused persons, complainants, act-section mappings, and arrest/surrender events; it also includes police organisational masters, court geography, case categories, gravity levels, and charge-sheet details. The PDF references at least one one-to-one occurrence-time table and one arrest-event-to-accused junction that are not fully rendered in the visible structure, so I treat those as valid design intent and normalise them explicitly below. turn0file0

The most important architectural principle is **separation of source-of-truth data from derived intelligence**. The relational operational database remains the auditable record of FIRs, accused, victims, arrests, and charge-sheets. Derived stores then serve specific purposes: a graph store for relationship traversal, a search index for lexical and semantic retrieval, a feature store for ML features, an object store for multimedia and PDFs, and a model registry/evidence ledger for reproducibility. This avoids turning the case database into a monolith that tries to handle transactional integrity, graph analytics, vector search, document storage, and AI lineage in one place. PostgreSQL's partitioning model is directly relevant for large fact tables and time-partitioned workloads, while Neo4j's property-graph model is better for multi-hop relationship discovery, and semantic search engines are better for meaning-based retrieval rather than keywords alone. ⁷

The second principle is **LLM as orchestrator, not authority**. The LLM should interpret user intent, choose tools, assemble results, explain reasoning, and draft summaries. It should not invent facts,

produce unaudited SQL, or make silent joins against sensitive data. Tool calling and JSON-schema-controlled outputs are the practical mechanisms that let the backend constrain the model and log exactly what it asked the data plane to do. That is what turns a conversational interface into an accountable investigative assistant instead of an opaque text generator. ⁸

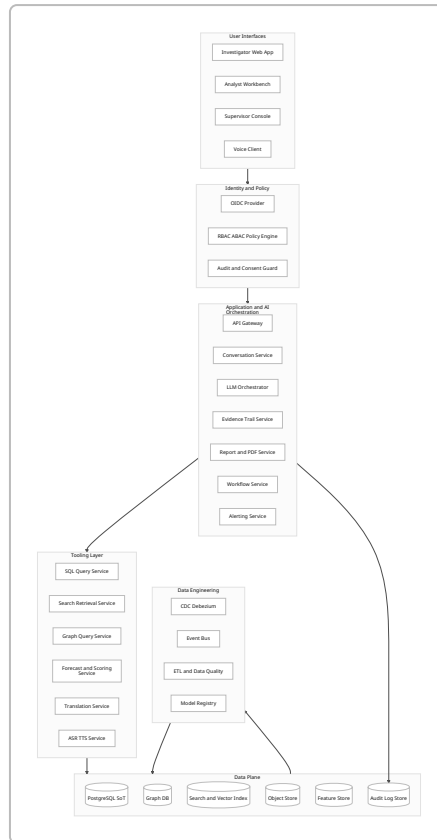
The third principle is **human decision rights stay with investigators and supervisors**. Forecasts, offender risk scores, similarity recommendations, and network centrality indicators must all be presented as decision-support artifacts with provenance and confidence markers. Because crime and victimisation data can be biased by reporting, case closure patterns, and enforcement intensity, the platform should explicitly separate descriptive analytics, triage recommendations, and operational actions. Where possible, use interpretable or semi-interpretable models for high-stakes outputs. ⁹

Reference architecture and deployment topology

The recommended target state is a **hybrid on-prem-first deployment** with four zones:

1. **Secure data zone** inside the state data centre or sovereign/cloud-approved environment: PostgreSQL, graph database, search cluster, object storage, CDC/event bus, identity, KMS/HSM integration, and SIEM pipeline.
2. **Application zone** for API gateway, chat orchestration, case workflow services, report generation, and analyst dashboards.
3. **ML inference zone** for self-hosted LLMs, ASR/TTS, embedding services, graph-ML jobs, and feature pipelines.
4. **Controlled external service zone** for optional commercial models through an egress gateway with policy enforcement, minimisation, redaction, and full request/response logging.

Kubernetes is the right substrate at the application and ML layers because it standardises control-plane/worker-node operations, autoscaling, and service isolation across both on-prem and cloud footprints. For identity, use OpenID Connect through Keycloak or an equivalent IAM service, and prefer authorization-code flow for user-facing applications instead of deprecated implicit/password-style shortcuts. ¹⁰



The deployment trade-off is straightforward. **Pure cloud** is faster to start and easier for managed search, observability, and commercial models, but it increases questions around data residency, vendor dependence, and legal defensibility for especially sensitive records. **Pure on-prem** gives stronger locality and potentially lower long-run inference cost at scale, but it demands MLOps maturity, GPU operations, and stronger in-house platform engineering. A **hybrid model** is the most balanced: keep all sensitive data and primary inference on-prem, but allow non-sensitive benchmarking, red-team testing, or burst-only model workloads in cloud. AWS and GCP both document pay-as-you-go VM pricing, while GCP also documents substantial committed-use discounts; those pricing mechanics are a useful basis for capacity planning, but not a substitute for a procurement quote. ¹¹

A good topology for HA production is **three logical environments**: production, pre-production, and isolated research/sandbox. In production, run stateless services across at least three worker groups, keep PostgreSQL in HA mode with synchronous replication for the primary region, replicate object storage and audit data to a DR site, and replicate graph/search asynchronously with tested reindex/rebuild runbooks. For the chat plane, maintain strict network egress policies so that only the LLM gateway can reach approved external endpoints. For the voice plane, keep ASR/TTS separate from the main chat orchestrator so that streaming load does not degrade high-priority investigative queries. ¹²

On-prem, cloud, and hybrid trade-offs

Decision area	On-prem first	Cloud first	Hybrid recommendation
Sensitive FIR and PII handling	Strongest residency and network isolation; best default for evidentiary data	Fastest launch, but stricter legal and data-transfer review needed	Keep the source-of-truth and audit plane on-prem; permit controlled cloud assistance only through a redacting gateway
LLM flexibility	Best with open-weight/self-hosted models; more MLOps burden	Broadest access to frontier APIs and rapid upgrades	Self-host primary summarisation/query models; optionally route low-risk tasks to commercial APIs
Voice	Whisper/Riva on local GPUs gives autonomy	Managed speech is fastest to pilot and maintains documented Kannada support	Managed speech for pilot; local ASR/TTS for production-sensitive units ¹³
Search and graph	Full control; more ops overhead	Managed services reduce toil	Self-host search/graph if legal constraints are strict; otherwise managed pre-prod is okay
Cost profile	High upfront CAPEX, lower marginal cost after scale	Low upfront CAPEX, recurring OPEX	Hybrid usually minimises legal risk and caps runaway API spend

Recommended technology baseline

The simplest defensible stack is: **PostgreSQL + pgvector** for transactional and lightweight vector operations; **Neo4j** for investigative graph traversal; **Elastic/OpenSearch-compatible search** for lexical and semantic case search; **MinIO/S3-compatible object storage** for media and PDFs; **Debezium + event bus** for CDC; **Kubernetes** for services; **Keycloak** for OIDC; **OpenTelemetry** for traces/metrics/logs; and a dedicated **LLM gateway** that mediates provider access, structured outputs, tool invocation, caching, and safety checks. PostgreSQL partitioning and pgvector's HNSW indexes are especially useful when the platform grows into multi-year, multi-district data volumes with embedding-assisted retrieval.

¹⁴

Backend services and end-to-end data flows

The backend should be organised as **domain-aligned microservices with a shared event backbone**, not as one giant API. The services below are the minimum practical decomposition for the requested capabilities.

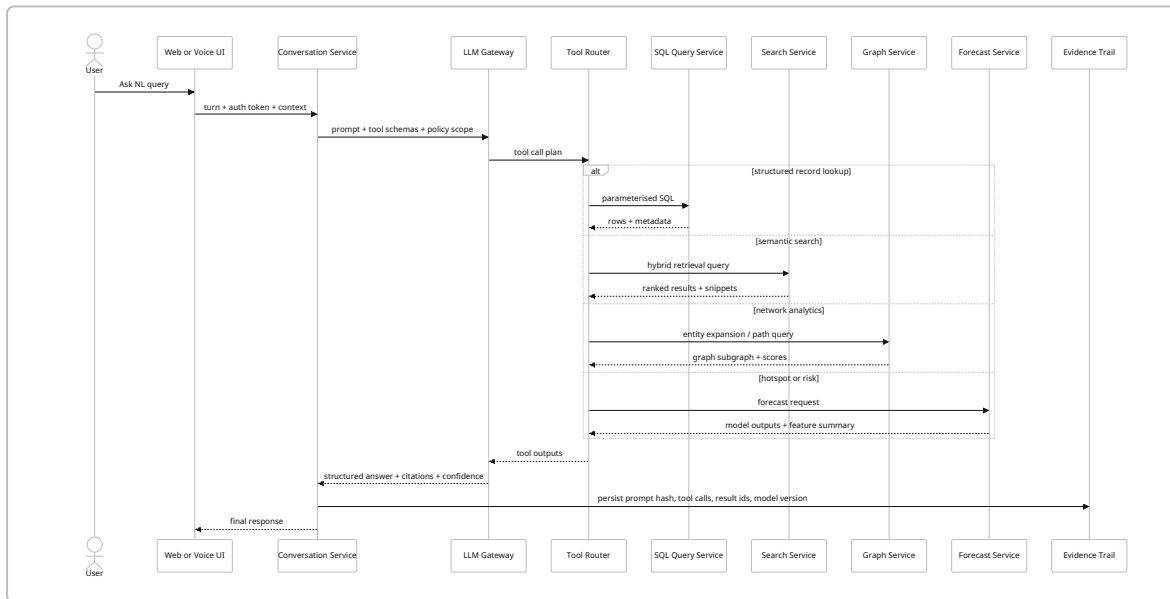
Service	Main responsibility	Key endpoints	Primary stores
Identity gateway	Session auth, token exchange, tenant/unit context	<code>/auth/login</code> , <code>/auth/refresh</code> , <code>/me</code>	OIDC/IAM

Service	Main responsibility	Key endpoints	Primary stores
Case service	CRUD and query over FIR/case records	<code>/cases</code> , <code>/cases/{id}</code> , <code>/cases/search</code> , <code>/cases/{id}/timeline</code>	PostgreSQL
Person-role service	Victim/accused/complainant CRUD; role views	<code>/cases/{id}/victims</code> , <code>/accused/{id}</code> , <code>/persons/canonical/{id}</code>	PostgreSQL + canonical person hub
Legal classification service	Act/section mappings, crime heads	<code>/acts</code> , <code>/sections</code> , <code>/cases/{id}/sections</code>	PostgreSQL
Investigation workflow service	Intake, tasking, lead management, closure	<code>/workflows/cases/{id}</code> , <code>/leads</code> , <code>/tasks</code> , <code>/closure-checks</code>	PostgreSQL
Evidence service	Chain of custody, media metadata, exhibit links	<code>/evidence</code> , <code>/cases/{id}/evidence</code> , <code>/exhibits/{id}</code>	PostgreSQL + object store
Graph intelligence service	Relationship graph build and query	<code>/graph/cases/{id}</code> , <code>/graph/entities/{id}/neighbors</code> , <code>/graph/community-detect</code>	Graph DB
Search and retrieval service	BM25 + semantic retrieval across cases/docs	<code>/search/cases</code> , <code>/search/docs</code> , <code>/search/hybrid</code>	Search index + vectors
Conversation service	Sessions, memory, turn history, PDF export	<code>/chat/sessions</code> , <code>/chat/turns</code> , <code>/chat/{id}/export/pdf</code>	PostgreSQL + object store
LLM orchestration service	Prompting, tool routing, citation assembly	internal only	model gateway
Forecast and scoring service	Hotspots, repeat-offender flags, risk and anomaly jobs	<code>/forecast/hotspots</code> , <code>/risk/offenders/{id}</code> , <code>/alerts</code>	feature store + model outputs
Translation and speech service	Kannada/English text + voice	<code>/translate</code> , <code>/asr/stream</code> , <code>/tts</code>	model endpoints
Audit and governance service	immutable events, access review, evidence ledger	<code>/audit/events</code> , <code>/evidence-trail/{id}</code>	append log store

This decomposition matters because the platform has **three very different runtime patterns**: interactive low-latency investigator queries, heavy analytical jobs, and continuous ingestion/synchronisation. Kubernetes and event-driven CDC provide the separation needed to scale these independently. OpenTelemetry should instrument every request path so that traces, metrics, and logs can be correlated across the gateway, orchestration layer, data tools, and downstream stores. ¹⁵

Natural-language query to response flow

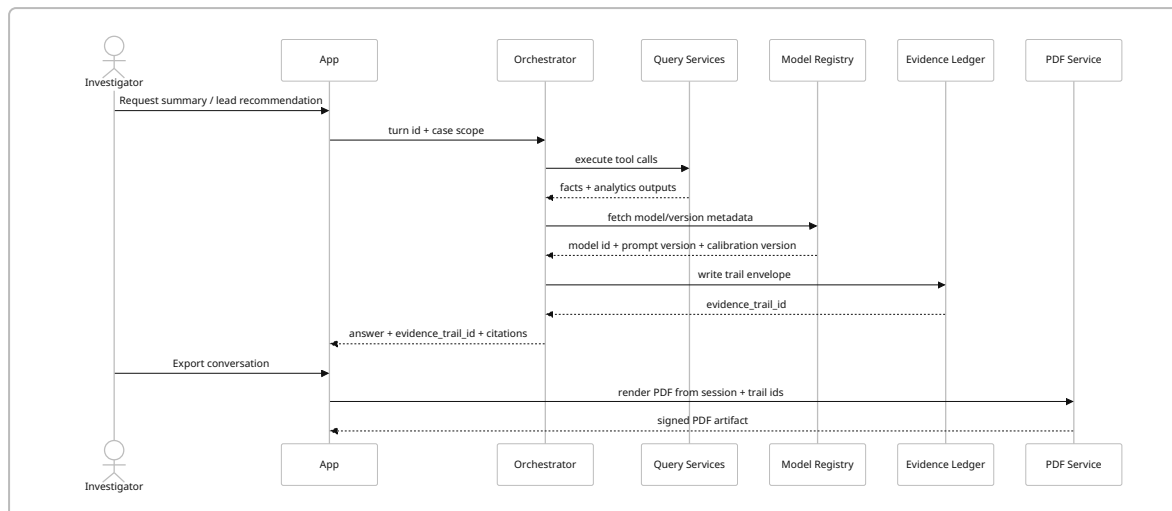
The conversational path should be strictly mediated. A user query enters the conversation service with the user's scope context (role, district, unit, current case, language, and active filters). The LLM receives only the minimum required case context plus tool schemas. It decides whether it needs SQL retrieval, graph expansion, search retrieval, or forecast lookup. The backend validates those calls, executes them, and returns structured tool results to the model. The model then synthesises an answer with citations, confidence cues, and an evidence-trail object that can be inspected or exported. Function calling and structured outputs are the enforcement mechanisms that keep this reliable. ⁸



A practical latency target is **2-5 seconds for simple single-tool lookups** and **5-12 seconds for graph-heavy or summarisation-heavy turns**. To hit that, apply prompt caching for long repeated system prefixes, keep tool schemas compact, and push all non-interactive jobs—mass summarisation, embeddings, nightly similarity scoring, backfills, exports—into asynchronous batch pipelines. Prompt caching and batch APIs are explicitly designed for this sort of cost/latency optimisation. ¹⁶

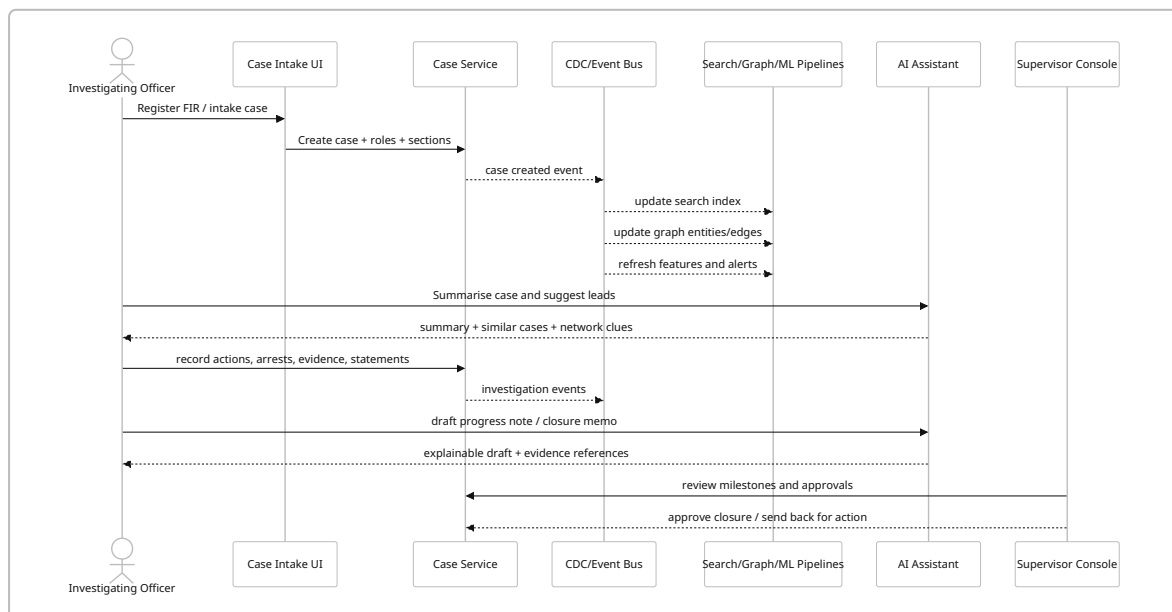
Evidence trail generation

Every answer that influences an investigation should produce a machine-readable **evidence trail**. This should include the user identity and role, request time, language, prompt template version, tool calls, SQL/Cypher/search requests, IDs of rows/documents returned, transformed derived features, model versions, output confidence, redaction decisions, and final citations. This makes supervisory review and later challenge handling much easier, and it aligns with the explainability requirement better than free-text “reasoning blurbs.” For high-stakes scoring, prefer inherently interpretable models or expose the decisive feature contributions directly rather than relying only on post-hoc explanations. ¹⁷



Investigator workflow from intake to closure

The workflow layer should not be an afterthought. It should turn a case from raw entry into a managed investigation with explicit stages, checklists, leads, and closure controls. That service is where conversational AI becomes operationally useful: summarising new FIRs, suggesting similar cases, surfacing repeated MOs, mapping known associates, flagging missing evidence, and preparing closure packages.



Data platform and ER design

Storage strategy

Use **polyglot persistence** deliberately. The transactional case database belongs in PostgreSQL because it needs ACID semantics, referential integrity, and strongly auditable updates. Partition large time-heavy tables such as `case_master`, `case_timeline_event`, `audit_log`, `financial_transaction`, and derived forecast outputs by month or quarter, typically with district or unit as a secondary dimension; PostgreSQL explicitly documents partitioning for very large tables and partition pruning for performance. Use pgvector only where local vector search is enough; for investigative search over long

documents and hybrid lexical/semantic ranking, a dedicated search store usually performs better operationally. Use Neo4j or another property graph for multi-hop association discovery because relationship-first traversals are its natural model. Use object storage for scanned documents, voice clips, photographs, and exported PDFs. ¹⁸

Core transactional schema based on the uploaded ER

The uploaded ER should remain the operational core. I recommend these base tables and keys, preserving the visible structure and normalising the two partially rendered design intents (`Inv_OccuranceTime` and `inv_arrestsurrenderaccused`) into explicit tables. The table names below are lower_snake_case equivalents of the uploaded design. [filecite turn0file0](#)

Table	Core fields	Indexes	Relati
case_master	case_master_id PK, crime_no UNIQUE, case_no, crime_registered_date, police_person_id FK, police_station_id FK, case_category_id FK, gravity_offence_id FK, crime_major_head_id FK, crime_minor_head_id FK, case_status_id FK, court_id FK, incident_from_date, incident_to_date, info_received_ps_date, latitude, longitude, brief_facts, created_at, updated_at	UNIQUE(crime_no), BTREE(police_station_id, crime_registered_date DESC), BTREE(case_status_id), GIST(geog_point), GIN(tsvector_brief_facts)	paren comp victim accus sectio arrest charg sheet eviden timeli
complainant_details	complainant_id PK, case_master_id FK, complainant_name, age_year, occupation_id FK, religion_id FK, caste_id FK, gender_id, mobile_no, address_id FK NULL, is_primary BOOLEAN	BTREE(case_master_id), GIN(trgm_name)	many comp per ca
victim	victim_master_id PK, case_master_id FK, victim_name, age_year, gender_id, victim_police BOOLEAN, person_canonical_id NULL	BTREE(case_master_id), GIN(trgm_name)	many per ca

Table	Core fields	Indexes	Relati
accused	accused_master_id PK, case_master_id FK, accused_name, age_year, gender_id, person_id_label, person_canonical_id NULL, is_known_habitual BOOLEAN, is_rowdy_sheeted BOOLEAN	BTREE(case_master_id), GIN(trgm_name), BTREE(person_id_label)	many accus case
arrest_surrender	arrest_surrender_id PK, case_master_id FK, arrest_surrender_type_id, arrest_surrender_date, arrest_surrender_state_id FK, arrest_surrender_district_id FK, police_station_id FK, io_id FK, court_id FK, accused_master_id FK NULL, is_accused, is_complainant_accused	BTREE(case_master_id, arrest_surrender_date DESC), BTREE(io_id)	one c many event
arrest_surrender_accused	arrest_surrender_id FK, accused_master_id FK, role_in_event, PRIMARY KEY(arrest_surrender_id, accused_master_id)	composite PK	explic to-ma where event involv multip accus
act	act_code PK, act_description, short_name, active	BTREE(active)	one a many sectio
section	act_code FK, section_code, section_description, active, PRIMARY KEY(act_code, section_code)	composite PK, BTREE(section_code)	sectio under
act_section_association	case_master_id FK, act_id, section_id, act_order_id, section_order_id, PRIMARY KEY(case_master_id, act_id, section_id)	composite PK, BTREE(act_id, section_id)	many mapp case
crime_head	crime_head_id PK, crime_group_name, active	BTREE(active)	paren crime head
crime_sub_head	crime_sub_head_id PK, crime_head_id FK, crime_head_name, seq_id	BTREE(crime_head_id, seq_id)	many heads one h

Table	Core fields	Indexes	Relati
crime_head_act_section	crime_head_id FK, act_code FK, section_code, PRIMARY KEY(crime_head_id, act_code, section_code)	composite PK	legal crime
chargesheet_details	csid PK, case_master_id FK, csdate, cstype, police_person_id FK, filing_court_id FK NULL, document_uri NULL	BTREE(case_master_id, csdate DESC)	many or rev possib
occurrence_time	case_master_id PK/FK, incident_from_date, incident_to_date, info_received_ps_date, latitude, longitude, place_text, geo_hash, police_beat_id NULL	PK, GIST(geog_point)	explicit to-one occur record in sou schen
case_timeline_event	event_id PK, case_master_id FK, event_type, event_ts, actor_employee_id NULL, notes, source_table, source_pk, importance	BTREE(case_master_id, event_ts), BTREE(event_type, event_ts)	derive timeli workf summ

The master/reference dimensions from the uploaded schema should remain conventional lookup tables: case_status_master, case_category, gravity_offence, occupation_master, religion_master, caste_master, court, district, state, unit, unit_type, rank, designation, employee, and optional gender_master and blood_group_master. They already map cleanly to the operational services and are important for both access control scopes and socio-demographic analytics. [filecite](#)turn0file0

Canonical entity and analytics extension

The operational ER is case-centric, but graph analytics works best when people, devices, accounts, phones, vehicles, addresses, and organisations become first-class entities. I therefore recommend adding a **canonical entity layer** without disrupting the case-entry model.

Table	Purpose	Key fields	Indexes
entity_profile	canonical node hub for person/account/device/vehicle/org/address/location	entity_id PK, entity_type, display_name, normalized_name, source_confidence, first_seen, last_seen, status	BTREE(entity_type, normalized_name), GIN(trgm_normalized_name)

Table	Purpose	Key fields	Indexes
entity_alias	aliases, spellings, transliterations	alias_id PK, entity_id FK, alias_text, lang_code, script_code, alias_type	GIN(trgm_alias_text)
entity_resolution_match	audited ER results	match_id PK, left_entity_id, right_entity_id, score, decision, decided_by, model_version, features_jsonb	BTREE(decision, score DESC)
case_entity_link	connect cases to canonical entities	case_master_id FK, entity_id FK, role_type, start_ts, end_ts, PRIMARY KEY(case_master_id, entity_id, role_type)	composite PK
relationship_edge	graph edges with provenance	edge_id PK, src_entity_id FK, dst_entity_id FK, edge_type, weight, first_seen, last_seen, case_count, properties_jsonb	BTREE(src_entity_id, edge_type), BTREE(dst_entity_id, edge_type), GIN(properties_jsonb)
community_detection_result	organisation/gang/community outputs	run_id, entity_id, community_id, algorithm, score	BTREE(run_id, community_id)
node_embedding	graph embeddings	entity_id, embedding, model_name, version, run_ts	vector/HNSW index
case_similarity	nearest-neighbour and MO similarity	case_master_id, similar_case_id, score, explanation_jsonb, run_id	BTREE(case_master_id, score DESC)

For financial and technical link analysis, add purpose-built entities and facts:

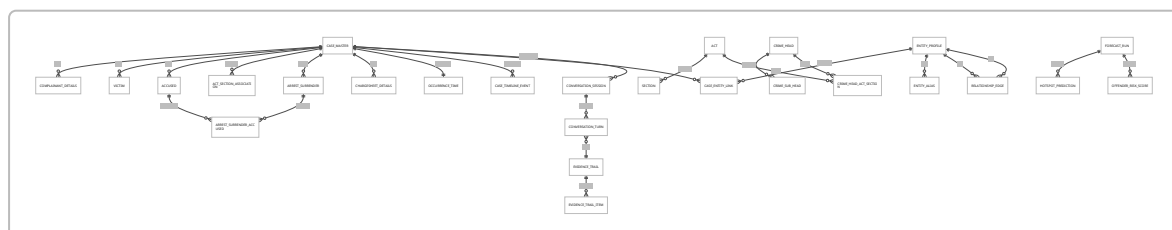
Table	Key fields	Notes
financial_account	account_id PK, account_type, masked_identifier, institution_name, kyc_entity_id NULL	never store full sensitive IDs in clear text unless lawful and encrypted
financial_transaction	txn_id PK, account_id FK, counterparty_account_id NULL, txn_ts, amount, currency, channel, location_id NULL, case_master_id NULL	monthly partitioned
device_identifier	device_id PK, imei_hash, imsi_hash, phone_hash, sim_hash, first_seen, last_seen	hash/tokenize at ingest unless exact raw values are legally required
vehicle_identifier	vehicle_id PK, registration_hash, vehicle_type, maker_model, first_seen, last_seen	supports case-linking without overexposing raw identifiers
address_master	address_id PK, normalized_text, lat, lon, geohash, district_id, state_id	geospatial join key
modus_operandi_master	mo_id PK, mo_code, mo_text, taxonomy_version, active	controlled MO taxonomy
case_mo_link	case_master_id, mo_id, confidence, source	extracted or manually assigned

For conversational AI, workflow, and auditability:

Table	Key fields	Notes
conversation_session	session_id PK, user_id, role_scope_jsonb, language_code, case_context_id NULL, started_at, ended_at	persistent context
conversation_turn	turn_id PK, session_id FK, speaker, utterance_text, translated_text NULL, asr_confidence NULL, tool_calls_jsonb, response_jsonb, evidence_trail_id NULL	full turn history
saved_report	report_id PK, session_id, case_master_id NULL, pdf_uri, hash, generated_by, generated_at	local PDF export support

Table	Key fields	Notes
evidence_trail	evidence_trail_id PK, request_hash, model_name, model_version, prompt_template_id, user_id, generated_at, final_confidence	core explainability artifact
evidence_trail_item	item_id PK, evidence_trail_id FK, item_type, source_system, source_ref, query_text, result_hash, order_no	SQL, search result, graph path, model feature, etc.
audit_log	audit_id PK, ts, user_id, action, resource_type, resource_id, decision, client_ip, device_id NULL, trace_id	append-only
alert	alert_id PK, alert_type, severity, entity_id NULL, case_master_id NULL, status, opened_at, closed_at	repeat crimes/ gang/hotspot warnings
forecast_run	run_id PK, model_name, version, feature_snapshot_id, run_ts, target_horizon	reproducibility
hotspot_prediction	run_id FK, location_cell_id, prediction_ts, risk_score, lower_ci, upper_ci, driver_summary_jsonb	map-serving table
offender_risk_score	score_id PK, entity_id FK, run_id, score, band, top_features_jsonb, review_required BOOLEAN	never use as autonomous decision output
feature_store_snapshot	snapshot_id PK, as_of_ts, feature_set_name, version, uri	training-serving consistency
model_registry	model_registry_id PK, model_name, provider, task_type, version, deployment_mode, approved_for_prod, approval_ref	controlled ML lifecycle

Core ER view



AI model stack and backend integration

Model roles and recommended choices

A production system should use a **portfolio** of models. The matrix below separates orchestration, bilingual understanding, graph learning, forecasting, and speech.

Function	Preferred production choice	Open/self-hosted alternative	Why this role exists
Conversation orchestration and reasoning	OpenAI GPT-5.x family or equivalent frontier reasoning model; Google Gemini Pro class; Anthropic Claude Opus/Fable class	Llama 3.1 70B or Mistral Large 3 / Small 4 for self-hosted secure deployments	Needed for intent decomposition, tool planning, summarisation, cross-source synthesis, follow-up handling, and multilingual chat. Meta documents Llama 3.1 at 8B/70B/405B with 128K context for multilingual and tool-use scenarios; Google and Anthropic position their flagship families for complex reasoning and agentic tasks. ¹⁹
Structured tool use	Models with function calling + strict JSON schemas	Same if self-host stack supports JSON tool calling	Needed so the LLM becomes an orchestrator over SQL/search/graph tools rather than a hallucinating narrator. Function tools and structured outputs directly support this pattern. ⁸
English/Kannada language understanding	MuRIL for Indian-language encodings; task-tuned IndicNER for entity tagging	Same	MuRIL is built specifically for Indian languages and transliterated text, and outperforms mBERT on XTREME tasks; IndicNER built on Naamapadam is a practical multilingual NER baseline. ²⁰
Translation	IndicTrans2	Managed translate API if policy allows	IndicTrans2 is designed for all 22 scheduled Indian languages and is a strong fit for English-Kannada bidirectional support. ²¹
Retrieval embeddings	multilingual E5 large for quality; base/small where latency matters	same	mE5 provides multilingual small/base/large options specifically for retrieval and RAG pipelines. ²²

Function	Preferred production choice	Open/self-hosted alternative	Why this role exists
Entity resolution	Gradient-boosted matcher over engineered features plus LLM-assisted review for ambiguous pairs	same	Practical ER needs deterministic blocking, explainability, and reviewability; do not use pure LLM matching on raw PII
Graph embeddings	node2vec first, GraphSAGE second	same	node2vec is efficient for linkage and similarity; GraphSAGE is inductive and works better for unseen nodes and evolving graphs. ²³
Temporal graph analytics	Temporal Graph Networks for mature graph streams	same	Use only after the base graph and features stabilise; TGNs handle dynamic event graphs better than static methods. ²⁴
Crime hotspot forecasting	Interpretable count/time models first; then Hawkes/self-exciting point processes and temporal graph models for mature deployments	same	Crime events exhibit spatiotemporal contagion and near-repeat effects, but forecasting must be bias-audited and advisory. ²⁵
Fraud/anomaly detection	classical anomaly + graph anomaly stack; heterophily-aware graph fraud models for mature financial link analysis	same	Organised-finance links often live in graphs and may be heterophilic; graph anomaly work is directly relevant here. ²⁶
Speech recognition	Whisper for offline/on-prem; Google or Azure managed STT for pilot; Riva for real-time GPU-serving	same	Whisper offers six model sizes with speed/accuracy trade-offs; Riva targets real-time speech AI; Google and Azure document Kannada support. ²⁷
Text-to-speech	Azure Speech or Google Gemini live/TTS where policy allows; Riva or other local TTS for sovereign deployments	same	Needed for investigator voice Q&A and accessibility; managed stacks already document low-latency live/TTS capabilities. ²⁸

Fine-tuning versus retrieval-augmented generation

For this project, the default should be **RAG plus tool use**, not broad generative fine-tuning. Fine-tune only where the task is stable, bounded, and measurable: NER over police narratives, Hindi/Kannada legal lexicon handling, entity-resolution scorers, document classification, redaction, and possibly MO tagging. Use retrieval and structured tool calls for everything that depends on current case data, roles/permissions, law sections, and investigation status. This is both safer and easier to audit. Function calling and structured outputs formalise the bridge between the model and backend tools, while multilingual embeddings and Indic language models handle the retrieval and extraction half of the job.

29

In practice, that means the orchestration prompt should follow four recurring patterns:

1. **Intent-to-tool routing prompt:** asks the model to classify the user request into SQL, graph, search, forecast, translate, or supervisor-only action.
2. **Schema-bounded extraction prompt:** used for NER, lead extraction, MO extraction, and summary JSON.
3. **Evidence-weaving prompt:** converts tool outputs into natural language while preserving citations and confidence.
4. **Challenge prompt:** asks the model to state what data is missing, what assumptions it made, and why its answer may be incomplete.

Put all shared instructions at the front of prompts so prompt caching can reduce latency and cost on repeated investigator workflows. For offline jobs such as nightly embeddings, similarity scoring, or large archive summarisation, prefer batch execution and queues. 16

Backend integration pattern

A clean integration contract for the LLM gateway looks like this:

```
POST /internal/llm/plan
{
  "user_context": {
    "user_id": "u-123",
    "role": "investigator",
    "district_scope": [44],
    "unit_scope": [3006],
    "language": "en-IN",
    "current_case_id": 982113
  },
  "conversation": {
    "session_id": "sess-789",
    "turn_id": "turn-21",
    "history_window": 8
  },
  "input": {
    "text": "Show similar robbery cases in the last 18 months near KR Puram and explain why."
  },
  "tools": ["sql_case_search", "hybrid_case_search", "geo_hotspot_lookup"],
```

```
"response_schema": "CaseRecommendationResponseV1"
}
```

A corresponding structured output schema should include `answer_text`, `cited_sources`, `tool_calls`, `confidence`, `assumptions`, and `follow_up_questions`. The backend then decides whether to let the model call tools in parallel, whether to redact fields before returning them to the model, and whether to require supervisor approval before showing outputs such as offender risk bands. Strict JSON-schema outputs are especially valuable here because they prevent malformed hand-offs between services. ⁸

Latency, throughput, caching, and batching

For capacity planning, think in three lanes:

Lane	Typical target	Design technique
Interactive chat	p95 under 8–10 seconds for most turns	prompt caching, short retrieval windows, prebuilt tool schemas, fast reasoning model for first pass
Investigator voice	partial transcript under 1 second, final answer in 4–10 seconds	streaming ASR/TTS, duplex session management, speech service isolation
Offline analytics	minutes to hours acceptable	queues, batch API, Spark/Flink jobs, graph rebuild windows, feature-store snapshots

Prompt caching can cut repeated-prefix latency and input cost materially when system prompts and tool definitions are long, and batch execution can cut asynchronous LLM job cost by roughly half while using separate higher-rate-limit capacity. These are not “nice to have” optimisations; for a statewide platform they become core economics. ¹⁶

Security, governance, operations, costs, and roadmap

Security and privacy architecture

Security should follow a **zero-trust, API-first, auditable** model. Every user and service gets short-lived tokens; every API checks scope; every high-risk action is logged; every exported artifact is attributable. Keycloak or an equivalent OIDC provider supports the required discovery, auth, token, introspection, certificate, and revocation endpoints. Use authorization-code flow for browser apps and client-credentials flow for service-to-service access; avoid deprecated password or implicit flows except for tightly constrained legacy cases. ³⁰

Map baseline controls to **NIST SP 800-53** families such as access control, audit, configuration, incident response, system/information integrity, and cryptographic protection, and apply **NIST zero-trust** principles to user, device, workload, and data paths. Defend APIs against broken object-level authorization, broken authentication, excessive data exposure, unsafe consumption of upstream APIs, and the rest of the OWASP API Security Top 10. ³¹

For Indian cyber obligations, the most concrete operational obligations in the current source set come from **CERT-In's 2022 directions**: incident reporting within six hours of noticing specified cyber incidents,

NTP synchronisation to approved time sources, secure log retention for 180 days within Indian jurisdiction, and a designated point of contact for CERT-In communications. The annex also explicitly lists attacks on AI/ML systems among reportable categories, which is highly relevant for this platform's model hosting and inference stack. ³²

For privacy, treat the platform as handling **highly sensitive operational personal data**. Minimise the data sent into model prompts, tokenise or hash identifiers where full values are not needed, encrypt data at rest and in transit, and keep the external-model path behind a redaction gateway. Differential privacy is useful for public or policy-facing aggregate dashboards and research extracts, but it is usually inappropriate for operational investigative records because it deliberately distorts case-level truth. The DPDP and IT Act frameworks are relevant, but exact retention, lawful basis, disclosure, and exemption issues should be validated with legal counsel before rollout. ³³

Explainability and model governance

Model governance should include a **model registry**, approval gates, evaluation packs, calibrated thresholds, shadow deployment, red-team cases, and rollback plans. NIST's AI governance direction is directly relevant here: the platform should treat validity, reliability, safety, security, accountability, and transparency as system properties, not just model properties. In a criminal-justice setting, that means versioned prompts, frozen training datasets for tuned extractors, audit trails for every deployed model, and mandatory human review for any score that could alter investigative priority or surveillance intensity. ³⁴

A practical explainability stack is:

Output type	Explainability mechanism
Case summary	citations to source tables/documents, timeline provenance, prompt/template ID
Network relationship	edge provenance, first/last seen, linked cases, edge confidence
Similar case recommendation	top matching features: MO, location, legal sections, temporal proximity
Hotspot forecast	location cell, horizon, confidence interval, key drivers
Offender score	risk band, top contributing features, review-required flag, prohibition on autonomous action

Logging, SIEM, monitoring, and backup

OpenTelemetry should instrument all services so that the platform emits correlated **traces, metrics, and logs**, and those should land in a central observability/SIEM stack. For a system this sensitive, security telemetry and application telemetry should be linked through trace IDs and principal IDs. That linkage materially improves forensic readiness, insider-threat detection, and post-incident reconstruction. ³⁵

Backups and DR should be designed per data class:

Data class	Backup pattern	DR target
PostgreSQL SoT	continuous WAL archiving + nightly fulls	RPO under 15 min, RTO under 2 hr for core case ops
Object store	versioned replication + immutable retention for evidence buckets	cross-site copy
Search index	snapshot and rebuild capability	rebuild acceptable if snapshots available
Graph DB	daily snapshots + event-replay rebuild	replay from CDC if needed
Audit/evidence ledger	append-only replication + immutability controls	highest retention priority
Feature store/model outputs	reproducible from snapshots and pipelines	lower recovery priority than SoT

Pen-test and assurance checklist

A meaningful pre-go-live test programme should include:

- API authorization testing for object-level and field-level access.
- IDOR/BOLA tests against case, person, evidence, export, and graph endpoints.
- Prompt-injection and tool-abuse testing against the LLM gateway.
- Data exfiltration tests on export, search, and generated summaries.
- SSRF and unsafe upstream-API consumption tests on model-provider adapters.
- Red-team tests on multilingual prompts, transliterated Kannada/English mixing, and toxic prompt chaining.
- Model jailbreak evaluation for “reveal hidden case details” and “ignore access policy” attempts.
- Graph query abuse tests to ensure a user cannot traverse into out-of-scope districts/units.
- Audit-log tamper tests and evidence-ledger consistency checks.
- Backup/restore drills and DR failover rehearsal.

These map directly to the API and cyber-risk concerns highlighted by OWASP and CERT-In. 36

Indicative cost model

Because exact pricing depends on concurrency, case volume, retention, GPU choice, and whether speech and frontier APIs are self-hosted or managed, the numbers below are **illustrative planning estimates**, not quotes.

Scenario	Indicative cost shape	Commentary
MVP cloud-assisted pilot	Low upfront cost, moderate recurring OPEX	Best when you need fast delivery and can tolerate managed-service dependency
MVP on-prem-heavy	Higher upfront spend, lower external API dependence	Better for sensitive workloads and long-run cost control

Scenario	Indicative cost shape	Commentary
Full statewide cloud-heavy	Lowest infrastructure operations burden but potentially highest recurring AI/search spend	risky if prompt volumes or speech usage spikes
Full statewide hybrid	Highest architectural complexity, best balance of sovereignty and elasticity	usually the most defensible public-sector target state

A workable rule of thumb is this: **cloud is cheaper to start, on-prem is cheaper to own once volumes stabilise**, and hybrid is the most governance-friendly for sensitive law-enforcement data. Public cloud pricing is pay-as-you-go by instance time or resource consumption, with committed-use discounts for longer commitments; asynchronous LLM jobs can also be materially cheaper when pushed into batch execution. ³⁷

Implementation roadmap

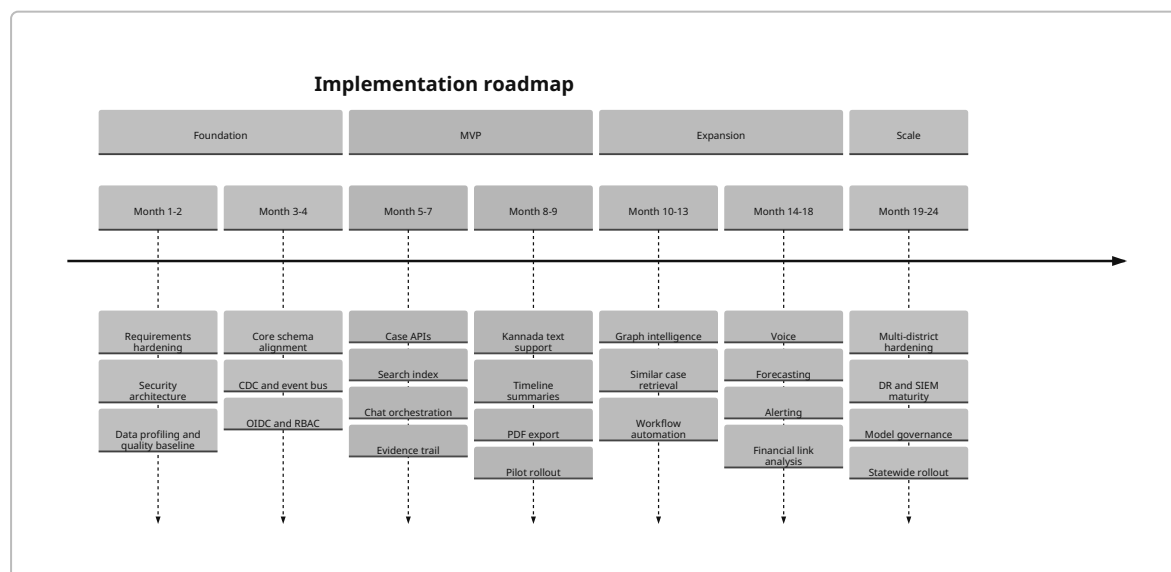
A realistic roadmap has two major releases.

MVP scope

MVP should include secure login, role-scoped chat, English/Kannada text interaction, FIR/case retrieval, victim/accused/complainant lookup, act-section search, case timelines, evidence trails, PDF export of conversations, hybrid search over case narratives, basic graph views, and descriptive hotspot dashboards. Voice can be added in MVP only if there is already speech infrastructure or a managed speech provider is acceptable. The MVP objective is not “full AI policing”; it is “explainable case intelligence with strong governance.” ³⁸

Full system scope

The full system adds streaming voice, mature entity resolution, financial link analysis, graph embeddings and community detection, forecasting, alerting, lead recommendation workflows, supervisor review queues, model registry, and cross-agency federation hooks. At that stage, the system becomes a governed crime-intelligence platform rather than an enhanced FIR search interface. ³⁸



Effort estimate

Workstream	MVP person-months	Full-system incremental person-months
Product, BA, domain design	4–6	6–10
Backend services and APIs	8–12	18–26
Data engineering and CDC	5–7	10–16
Search and graph platform	4–6	10–16
Frontend and investigator UX	5–7	8–12
AI orchestration and retrieval	4–6	10–16
ML forecasting / graph ML / ER	0–3	12–20
MLOps, observability, DevSecOps	4–6	8–12
Security, audit, compliance, testing	4–5	8–12
Total	30–45	60–95

That yields a **full-programme total of roughly 90–140 person-months**, with the lower end assuming strong existing data quality, existing IAM/Kubernetes foundations, and limited multilingual voice scope.

Open questions and limitations

Some important items remain assumptions because the available inputs do not fully specify them.

The uploaded ER diagram is clear on the core operational model, but it does not fully expose every occurrence-time or arrest-junction detail in machine-readable form, so those parts are normalised in this report as explicit tables inferred from the relationship matrix. [filecite turn0file0](#)

The legal/compliance section should be read as **technical governance guidance, not legal advice**. The IT Act, CERT-In directions, and the DPDP framework are clearly relevant, but precise applicability to state law-enforcement record processing, exemptions, inter-agency sharing, and retention must be validated by counsel and departmental policy owners. [39](#)

The cost section is intentionally approximate. Public cloud providers document billing mechanics and discounts, but state-scale totals depend heavily on concurrency, retention, case volume, and whether frontier-model usage is concentrated in synchronous chat versus low-cost asynchronous pipelines. [37](#)

Finally, forecasting and offender scoring require especially careful governance. The literature supports spatiotemporal crime modelling and graph-based relationship learning, but it also shows clear risks from reporting bias, feedback loops, and opaque models in criminal-justice contexts. That is why the architecture above treats these functions as supervised decision support rather than autonomous action systems. [9](#)

- 1 10 12 15 <https://kubernetes.io/docs/concepts/overview/components/>
<https://kubernetes.io/docs/concepts/overview/components/>
- 2 8 29 <https://developers.openai.com/api/docs/guides/function-calling>
<https://developers.openai.com/api/docs/guides/function-calling>
- 3 <https://platform.openai.com/docs/models>
<https://platform.openai.com/docs/models>
- 4 25 <https://arxiv.org/abs/1708.03579>
<https://arxiv.org/abs/1708.03579>
- 5 31 <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
<https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- 6 <https://debezium.io/documentation/reference/stable/architecture.html>
<https://debezium.io/documentation/reference/stable/architecture.html>
- 7 14 18 <https://www.postgresql.org/docs/current/ddl-partitioning.html>
<https://www.postgresql.org/docs/current/ddl-partitioning.html>
- 9 <https://arxiv.org/abs/2102.00128>
<https://arxiv.org/abs/2102.00128>
- 11 37 <https://aws.amazon.com/ec2/pricing/on-demand/>
<https://aws.amazon.com/ec2/pricing/on-demand/>
- 13 <https://docs.nvidia.com/deeplearning/riva/user-guide/docs/overview.html>
<https://docs.nvidia.com/deeplearning/riva/user-guide/docs/overview.html>
- 16 <https://developers.openai.com/api/docs/guides/prompt-caching>
<https://developers.openai.com/api/docs/guides/prompt-caching>
- 17 <https://arxiv.org/abs/1811.10154>
<https://arxiv.org/abs/1811.10154>
- 19 <https://ai.meta.com/blog/meta-llama-3-1/>
<https://ai.meta.com/blog/meta-llama-3-1/>
- 20 <https://arxiv.org/abs/2103.10730>
<https://arxiv.org/abs/2103.10730>
- 21 <https://github.com/AI4Bharat/IndicTrans2>
<https://github.com/AI4Bharat/IndicTrans2>
- 22 <https://arxiv.org/abs/2402.05672>
<https://arxiv.org/abs/2402.05672>
- 23 <https://arxiv.org/abs/1607.00653>
<https://arxiv.org/abs/1607.00653>
- 24 38 <https://arxiv.org/abs/2006.10637>
<https://arxiv.org/abs/2006.10637>
- 26 <https://arxiv.org/abs/2106.07178>
<https://arxiv.org/abs/2106.07178>
- 27 <https://github.com/openai/whisper>
<https://github.com/openai/whisper>
- 28 <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/language-support>
<https://learn.microsoft.com/en-us/azure/ai-services/speech-service/language-support>

- 30 <https://www.keycloak.org/securing-apps/oidc-layers>
<https://www.keycloak.org/securing-apps/oidc-layers>
- 32 39 https://www.cert-in.org.in/PDF/CERT-In_Directions_70B_28.04.2022.pdf
https://www.cert-in.org.in/PDF/CERT-In_Directions_70B_28.04.2022.pdf
- 33 https://en.wikipedia.org/wiki/Information_Technology_Act%2C_2000
https://en.wikipedia.org/wiki/Information_Technology_Act%2C_2000
- 34 <https://www.nist.gov/itl/ai-risk-management-framework>
<https://www.nist.gov/itl/ai-risk-management-framework>
- 35 <https://opentelemetry.io/docs/concepts/observability-primer/>
<https://opentelemetry.io/docs/concepts/observability-primer/>
- 36 <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
<https://owasp.org/API-Security/editions/2023/en/0x11-t10/>